



Análise e Desenvolvimento de Sistemas **ENGENHARIA DE**
SOFTWARE ÁGIL APLICADA RELATÓRIO DE AULAS
PRÁTICAS

Nome: Vitória Freitas

RA: 2523279

Polo de matrícula: Carapicuíba - Vila Dirce

Local da realização da Aula Prática: Alphaville - Santana de Parnaíba

Ano da postagem: 2026

RELATÓRIO 1: Modelagem de Processos e Qualidade com Draw.io

Resumo teórico

Qual a utilização da modelagem de processos?

A modelagem de processos tem como objetivo servir às organizações uma linguagem comum na qual elas podem usar para otimizar e entender fluxos de trabalhos, se consegue fazer isso pois, a modelagem de processos é a prática de criar representações visuais com base nos dados dos processos de negócios.

Explique cada item da estrutura da ISO 9241.

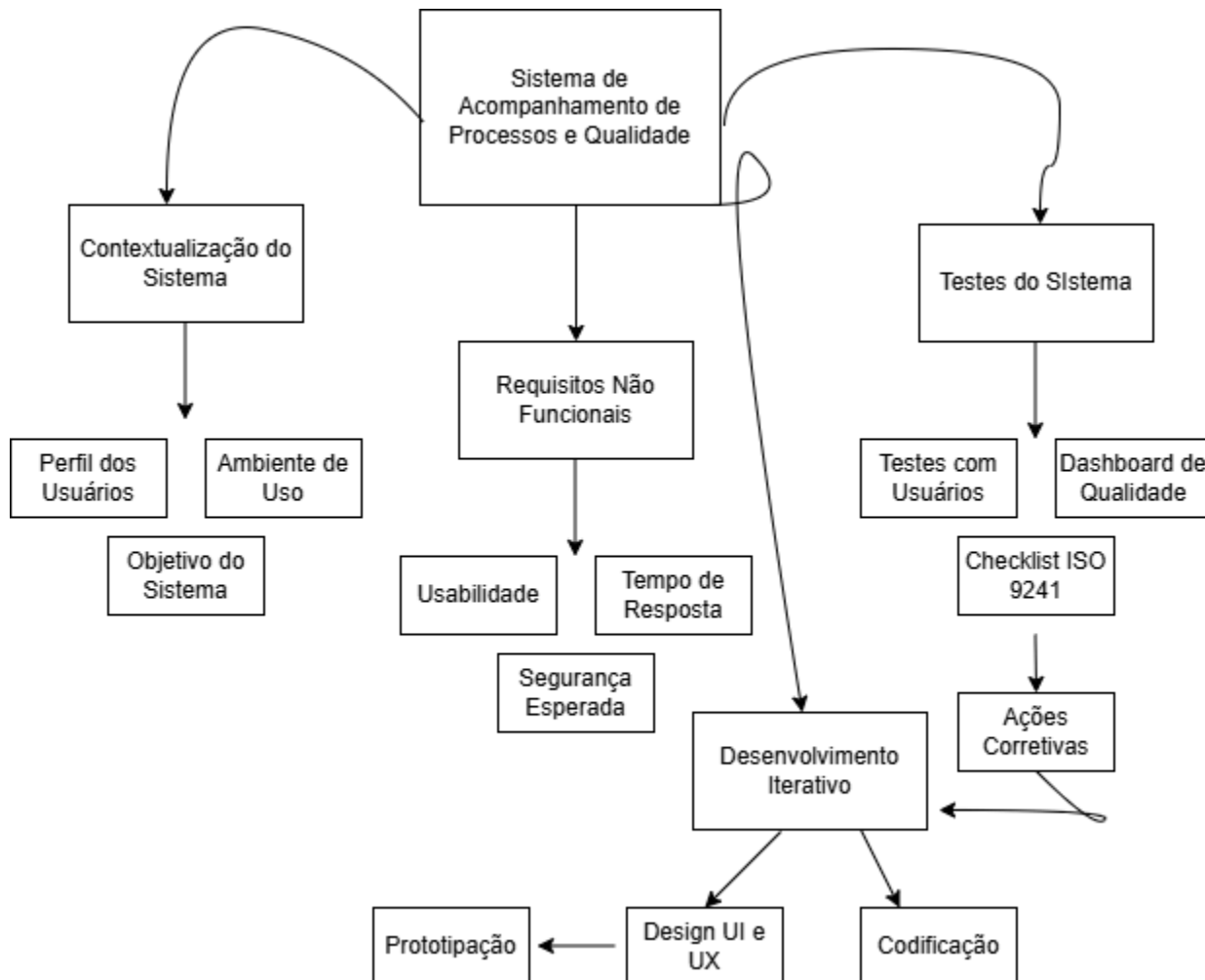
A ISO 9241 é uma norma da Organização Internacional de Normalização (ISO) que abrange a ergonomia da interação humana e do sistemas. Ela é dividida em diversas partes, a parte 1 é uma introdução geral da norma, a parte 2 trata sobre o projeto de tarefas para o trabalho com sistemas computacionais, as partes 3 e 9 tratam das características físicas dos computadores, a parte 11 a 19 e a 110 abordam a usabilidade do software.

Como será desenvolvido o protótipo da aplicação com Draw.io, para o controle de processos da ISO 9241.

O desenvolvimento do protótipo da aplicação com Draw.io, para o controle de processos da ISO 9241 foi construído em base nas partes da ISO 9241, criamos um diagrama para um sistema de acompanhamento de processos e qualidade, inserindo testes de usuários, checklist da ISO 9241 para a asseguuração que a norma está sendo cumprida pelo sistema que está sendo acompanhado.

Prototipagem da aplicação com Draw.io

Figura 1 - Imagem do diagrama do sistema feito pelo Draw.io



Draw.io é um software gratuito para criação de diagramas, fluxogramas e UML, sendo muito fácil de usar.

Reflexões finais e análise crítica

A atividade foi interessante de fazer, pois demonstra a importância da modelagem do processo de desenvolvimento de software, tendo como dificuldade entender as necessidades e objetivos do negócio antes da criação dos diagramas.

Com a modelagem do processo de desenvolvimento de software podemos entender, quais necessidades do usuário o sistema precisa entender, servindo como uma linguagem comum para diversos setores da empresa possam entender o sistema que será desenvolvido, assim trazendo melhorias a produtividade. Em suma, com essa

aula prática aprendemos a importância dessas modelagens na engenharia de software.

Referências

<https://www.ibm.com/br-pt/think/topics/process-modeling>

https://en-wikipedia-org.translate.goog/wiki/ISO_9241?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

RELATÓRIO 2: Engenharia de Requisitos e Análise de Risco com FreeMind

Resumo teórico

A fase do desenvolvimento de software em que ocorre a coleta e análise do risco.

O ciclo de vida de desenvolvimento de software é onde ocorre o desenvolvimento em fases distintas, a fase que acontece a coleta e análise de risco é nas primeiras duas fases, a de planejamento, na qual as atividades principais é identificar o escopo, as metas e os requisitos do projeto e a entrega é o plano inicial do projeto, a segunda é a análise, nela as atividades principais é coletar e revisar dados sobre os requisitos do projeto e a entrega é documentação de requisitos detalhada.

O conceito sobre os requisitos RU, RNF, RF e RS e riscos associados.

O grande risco dos requisitos, é coleta deles de maneira mal feita, seja elas mal organizados, mal expressos ou requisitos desnecessários e dificuldade de lidar com requisitos frequentemente mutáveis, podendo ser uma razão para o fracasso de um software.

Os requisitos funcionais são aqueles sobre o que o sistema deve fazer, suas funções, eles se preocupam com a funcionalidade do sistema, como por exemplo:

- [RF001] O Sistema deve emitir um certificado para os usuários.
- [RF002] O Sistema deve ter página do usuário e página do administrador.
- [RF003] O Sistema deve permitir ao usuário administrador editar a quantidade de itens disponíveis no estoque.

Os requisitos não funcionais são aqueles que qualificam os requisitos funcionais, como a qualidade do software, performance, usabilidade, confiabilidade, robustez e etc, exemplos de requisitos não funcionais:

- [RNF001] O Sistema deve emitir o certificado em menos de 2 segundos.
- [RNF002] O Sistema deve negar acesso a edição do estoque ao

usuário que não seja o administrador.

Os requisitos de usuário descrevem as necessidades do cliente, sendo escrito em linguagem natural e os requisitos do sistema são descrições mais detalhadas que os requisitos do usuário, são usados pela equipe de desenvolvimento, possuindo informações técnicas sobre o projeto.

Quais os benefícios oferecidos pelos mapas mentais e como isso auxilia as equipes no desenvolvimento do software.

Os mapas mentais ajudam na concentração, são melhores para absorção de informações, assim facilitando a compreensão e visão geral do projeto, melhorando a produtividade da equipe.

Estudo de caso

O sistema de software escolhido foi um ERP da TOTVS para uma empresa de vendas. A TOTVS é um empresa que desenvolve softwares para diversas áreas (como: rh, finanças, vendas e etc), o sistema escolhido foi o ERP, o ERP é um software que integra todos os departamentos de uma empresa.

Construção da tabela de requisitos de risco

Requisitos Funcionais	Requisitos Não Funcionais
[RF001]: O sistema deve permitir criar, acompanhar e finalizar pedidos de vendas, incluindo emissão de nota fiscal.	[RNF001]: O sistema deve ter disponibilidade mínima de 99,5% em horário comercial, com janela de manutenção programada aos domingos.
[RF002]: O sistema deve atualizar automaticamente o saldo de estoque a cada venda, compra ou devolução.	[RNF002]: A consulta de um pedido de venda ou extrato de cliente deve ser exibida até 3 segundos.
[RF003]: O sistema deve integrar as vendas ao contas a	[RNF003]: Sistema deve implementar acesso por perfil

receber e as contas a pagar.	(vendedor, gerente, financeiro).
[RF004]: O sistema deve armazenar histórico de compras por cliente, permitir análise de limite de crédito.	[RNF004]: A interface para lançamento de pedidos deve ser concluída em no máximo 5 cliques, com campos de preenchimento automático a partir do CNPJ do cliente.
[RF005]: O sistema deve gerar relatórios como: faturamento por período, produto mais vendido e desempenho por vendedor.	[RNF005]: O ERP deve expor APIs RESTful para integração com e-commerce, marketplaces (ex: Magalu, Mercado Livre) e sistemas de automação de marketing (ex: RD Station).

Diagrama dos requisitos com mapa mental

Figura 2 - Mapa Mental dos requisitos do sistema



Esse mapa mental foi criado pelo FreeMind, que é uma ferramenta gratuita para criação de mapas mentais para diversos projetos, ajudando na criação de projetos e na produtividade da equipe.

Reflexões finais

Os mapas mentais ajudam na compreensão dos requisitos funcionais, não funcionais, requisitos do usuário e do sistema, facilitando a criação de projetos para diversas equipes no desenvolvimento de software, aumentando a produtividade da empresa. Com eles podemos ter uma melhor comunicação entre a equipe de desenvolvimento e os clientes, usando uma linguagem natural

para todos possam entender o que o sistema terá que fazer e as funções que ele irá cumprir, o que é essencial para projeção do sistema.

Ferramentas como FreeMind, são fundamentais para a criação de mapas mentais, sendo uma ferramenta gratuita e com interface intuitiva, ela facilita o acesso a ela, com isso diversas pessoas e empresas podem utilizar sem ter restrição de acesso.

Os resultados obtidos com essa atividade, foi uma melhor compreensão sobre levantamentos de requisitos e sua importância, junto com a relevância da utilização de mapas mentais para a engenharia de software, tendo em vista, suas dificuldades como entender as reais necessidades do usuário e os pontos de melhoria, como conhecer de uma maneira mais aprofundada as ferramentas que facilitam esse processo de desenvolvimento e melhoram o nosso resultado ao desenvolver um software.

Referências

<https://www.ibm.com/br-pt/think/topics/sdlc>

<https://www.devmedia.com.br/introducao-a-requisitos-de-software/29580>

https://homepages.dcc.ufmg.br/~figueiredo/disciplinas/aulas/req-usuario-sistema_v01.pdf

<https://online.pucrs.br/blog/mapas-mentais>

<https://www.totvs.com/>

<https://contaazul.com/blog/erp/>

RELATÓRIO 3: Ciclo PDCA Aplicado e Tabela PERT com GanttProject

Resumo teórico

Para que a metodologia PDCA é usada, sua ligação com a ISO 9001 e aplicação na gestão de tarefas de metodologias ágeis.

A metodologia PDCA (Plan, Do, Check, Act) é eficaz para a melhoria contínua de processos e gestão de qualidade nas empresas, tanto que a norma ISO 9001 recomenda o seu uso, por facilitar a implementação de um Sistema de Gestão da Qualidade (SGQ) e por identificar riscos e oportunidades. O PDCA traz uma garantia que os processos serão avaliados e ajustados de uma maneira sempre constante, assim facilitando ajustes para prever e corrigir erros.

O conceito associado à tabela PERT

Um diagrama de PERT é uma ferramenta de gestão de projetos, com o objetivo de análise de tarefas envolvidas na conclusão de um projeto.

Como funciona o cronograma e o kanban na gestão de projetos ágeis.

O cronograma e o kanban impulsionam a eficiência ao organizar a progressão das tarefas, através de fluxos de trabalho visualizados, permitindo o acompanhamento transparente do progresso.

O ciclo PDCA

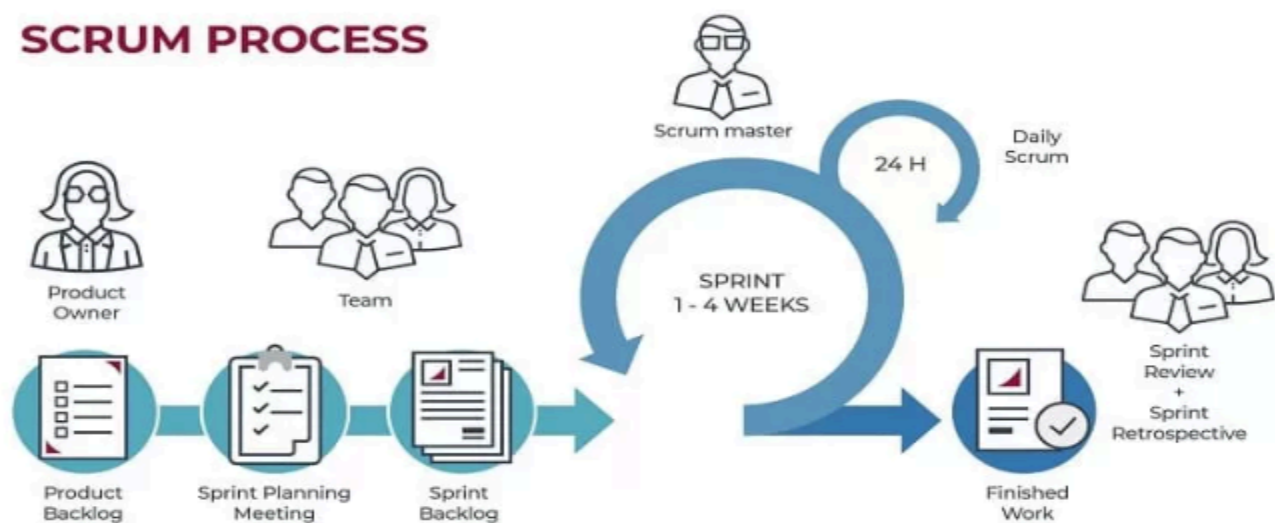
O PDCA (Plan, Do, Check, Act) é uma ferramenta de qualidade de quatro fases, usada para a solução de problemas, controle e melhoria de processos e produtos.

- Plan: Nessa etapa são definidos metas e objetivos do ciclo, gerando um plano de ação.
- Do: Nessa etapa o plano de ação definido na anterior é colocado em prática.
- Check: Essa fase deve iniciar junto com a implementação do Plano de Ação, pois aqui se faz o acompanhamento dos resultados, monitorando cada atividade do plano de ação.
- Act: Nesta ocorre correções, caso tenha diferenças identificadas, se não houver, acontece a padronização e a conclusão do plano com ações corretivas sobre processos de planejamento, execução e auditoria.

Metodologia ágil: Scrum

O Scrum é uma estrutura ágil que organiza o trabalho em sprints com tempo definido. As funções principais incluem o proprietário do produto, Scrum Master e a equipe de desenvolvimento.

Figura 3 - Scrum



Fonte:
<https://giovanidacruz.com.br/introducao-a-metodologia-scrum-uma-abordagem-agil-para-gerenciamento-de-projetos/> (2026)

Tabela PERT

A metodologia PERT é muito utilizada na Gestão de Projetos, ajudando a estimar prazos de duração de projetos, evita que atrasos ocorra e destaca tarefas mais importantes, o que prejudicaria o planejamento.

Etapas do PDCA	ID	Tarefa	Dependências	Períodos (dias)
Plan	1	Identificar o problema	-	3
	2	Analisar causas	1	4
	3	Definir metas e plano de ação	2	5
Do	4	Implementar ações	3	7

		planejadas		
Check	5	Coletar dados e monitorar resultados	4	4
	6	Comparar resultados com metas	5	2
Act	7	Padronizar soluções eficazes	6	3
	8	Ajustar ou corrigir falhas	6	4

Legenda:

Dependências: indica a ID da tarefa que precisa ser concluída antes.

Períodos: estimativa de tempo por tarefa.

Construção do cronograma e rede de cartões (Kanban)

O Kanban é um framework Ágil que visualiza o trabalho, limitando o trabalho em andamento e promovendo melhoria continuamente e o uso dele com cronogramas é extremamente eficaz para rastrear tarefas e identificar gargalos que prejudicam o desenvolvimento do projeto, assim otimizando os ciclos de entrega.

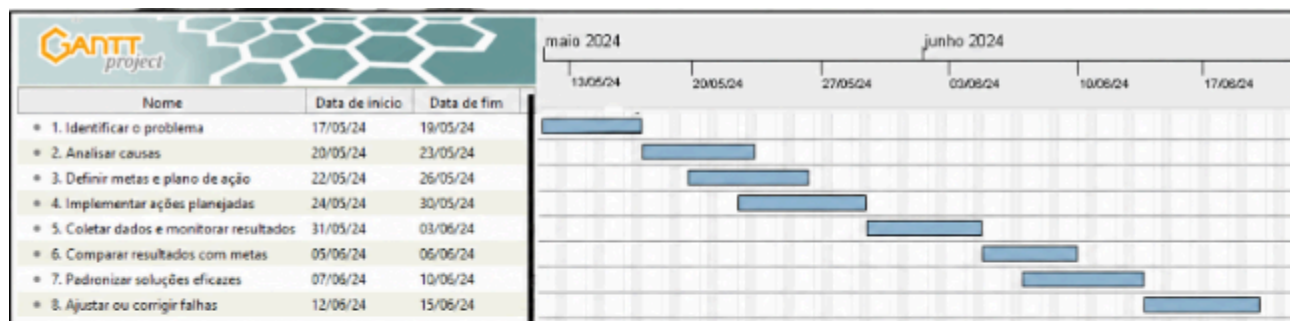


Figura 1 – Cronograma do projeto "Gantt" (GanttProject).

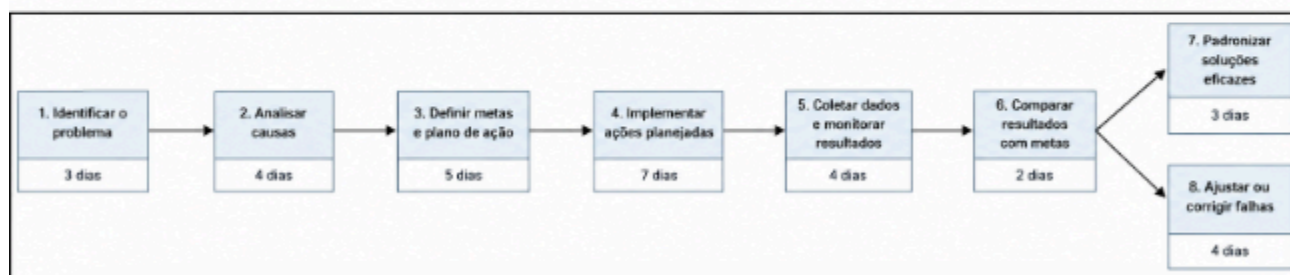


Figura 2 – Mapa da rede kanban pelo "Gráfico PERT" (GanttProject).

Reflexões finais

Com o ciclo PDCA podemos distribuir as tarefas dos sprints do Scrum em 4 etapas diferentes, etapa de planejamento, de fazer o que foi planejado, de verificação e de ação.

Elementos principais para a construção da tabela PERT seria a identificação dos problemas que precisam ser resolvidos e o tempo de duração para a resolução dele.

Com cronograma e da rede de cartões no acompanhamento das atividades do desenvolvimento conseguimos ter uma maior eficiência durante o processo de planejamento e ação dos projetos, eles impulsionam a produtividade e comunicação da equipe ao trazer uma visualização para os processos de desenvolvimento.

As principais funcionalidades da aplicação GanttProject para a construção de cronogramas e rede de cartões são o monitoramento de cada atividade, mostrando quando uma inicia e quando termina, com isso, uma atividade que tem dependência de outra, apenas inicia quando a que ele necessita pronta, já tenha finalizado.

Referências

<https://blogdaqualidade.com.br/metodologia-pdca-e-a-iso-9001/>

<https://www.jobconvo.com/blog/ciclo-pdca/>

<https://www.lucidchart.com/pages/pt/guia/diagrama-pert>

<https://www.atlassian.com/br/agile/kanban>

<https://www.treasy.com.br/blog/ciclo-pdca/>

<https://www.educamaisbrasil.com.br/educacao/dicas/metodologia-pert-o-que-e-e-como-funciona>

RELATÓRIO 4: MPS.BR, SWOT e Kanban Integrados com ChatGPT

Resumo teórico

Níveis de maturidade do modelo MPS.BR.

O modelo de Melhoria de Processo de Software Brasileiro (MPS.BR) é uma iniciativa para aumentar a competitividade das empresas brasileiras. O modelo define 7 níveis de maturidade, esses são:

- G - Parcialmente Gerenciado
- F - Gerenciado
- E - Parcialmente Definido
- D - Definido
- C - Totalmente Definido
- B - Gerenciado Quantitativamente
- A - Em otimização

Porque a matriz SWOT é útil como ferramenta de análise estratégica.

A matriz SWOT é uma ferramenta capaz de mapear os pontos fortes e os pontos de melhoria de uma organização. Ela facilita o diferenciamento de obstáculos internos e externos, bem como qualidades e oportunidades, além de propiciar estabelecimento de metas.

O significado de SWOT é S de strengths (forças), W de weaknesses (fraquezas), O de opportunities (oportunidades) e T de threats (ameaças).

Com a matriz SWOT a empresa pode utilizá-la para definir um planejamento estratégico, com a ferramenta fornecendo uma análise que leva em consideração os prós e os contras de determinada situação.

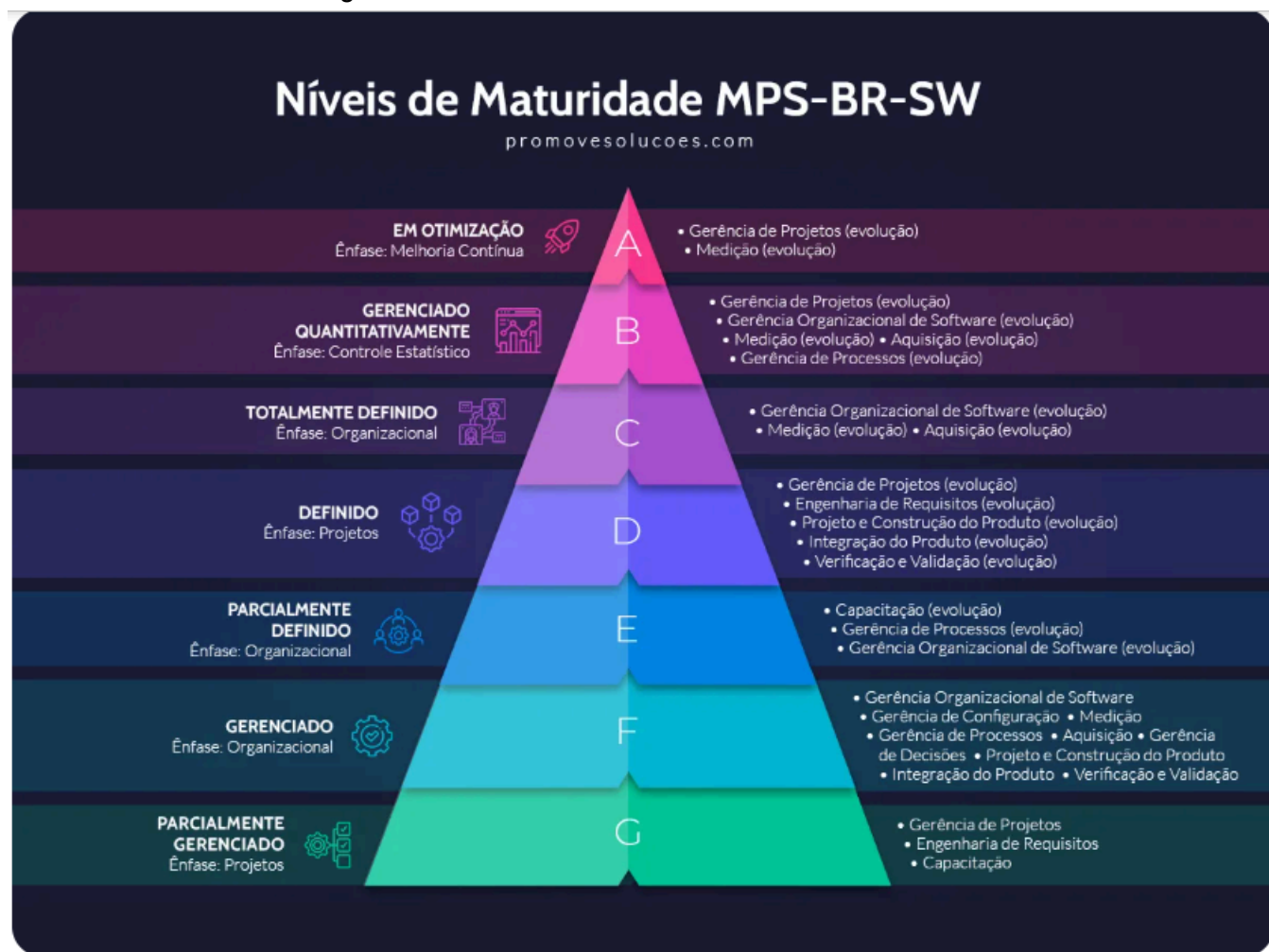
Como funciona o kanban na gestão de projetos ágeis.

O Kanban é um framework Ágil que visualiza o trabalho, impulsionando a melhoria contínua, definindo limites de trabalho e usando métricas, como tempo de ciclo. O kanban promove o aprimoramento do foco, eficiência e adaptabilidade da equipe.

Os benefícios em utilizar suporte inteligente no desenvolvimento.

Entre os diversos benefícios de utilizar suporte inteligente no desenvolvimento, podemos citar o aumento de produtividade e facilitação durante algumas tarefas do desenvolvimento.

Figura 5 - Níveis de Maturidade MPS-BR-SW



Fonte:

<https://promovesolucoes.com/quais-sao-os-niveis-de-maturidade-do-mps-br/>

Matriz SWOT

Com a matriz SWOT podemos mapear os pontos fortes e os pontos de melhoria de um projeto, facilitando o diferenciamento de obstáculos internos e externos, bem como qualidades e oportunidades.

Figura 6 - Matriz SWOT



Fonte:

<https://radar.ibegesp.org.br/como-a-analise-swot-pode-melhorar-a-estrategia-de-uma-organizacao/>

Kanban

O Kanban é um framework Ágil que impulsiona a melhoria contínua, promovendo o aprimoramento do foco, eficiência e adaptabilidade da equipe. Sendo uma ferramenta essencial para o desenvolvimento ágil.

ChatGPT

Comentários do ChatGPT:

Criei um painel completo em HTML integrando:

- MPS.BR (controle de processos)
- Análise SWOT
- Quadro Kanban
- Integração com API do ChatGPT

O código já possui:

- HTML
- CSS
- JavaScript
- Estrutura responsiva
- Exemplo de consumo da API da OpenAI

Você só precisa substituir `SUA_CHAVE_API` pela sua chave da API da OpenAI para funcionar.

Código gerado:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Painel Integrado - MPS.BR, SWOT e Kanban</title>

  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
      font-family: Arial, Helvetica, sans-serif;
    }

    body {
      background: #f4f7fb;
      color: #333;
      padding: 20px;
    }
  </style>
</head>
<body>
```



```

h1 {
  text-align: center;
  margin-bottom: 20px;
  color: #0d47a1;
}

.container {
  display: grid;
  grid-template-columns: 1fr;
  gap: 25px;
}

.card {
  background: white;
  border-radius: 12px;
  padding: 20px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

.card h2 {
  margin-bottom: 15px;
  color: #1565c0;
}

/* SWOT */
.swot-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 15px;
}

.swot-item {
  padding: 15px;
  border-radius: 10px;
  min-height: 150px;
}

.forcas { background: #d0f0d0; }
.fraquezas { background: #ffe0b2; }
.oportunidades { background: #bbdefb; }
.ameacas { background: #ffcdd2; }

```

```

textarea {
  width: 100%;
  margin-top: 10px;
  padding: 10px;
  border-radius: 8px;
  border: 1px solid #ccc;
  resize: vertical;
  min-height: 80px;
}

/* MPS */
table {
  width: 100%;
  border-collapse: collapse;
}

table th,
table td {
  border: 1px solid #ddd;
  padding: 10px;
  text-align: center;
}

table th {
  background: #1976d2;
  color: white;
}

select {
  padding: 6px;
  border-radius: 6px;
  border: 1px solid #ccc;
}

/* Kanban */
.kanban {
  display: flex;
  gap: 20px;
  flex-wrap: wrap;
}

```

```

.column {
  flex: 1;
  min-width: 250px;
  background: #eef3f9;
  padding: 15px;
  border-radius: 10px;
}

.column h3 {
  margin-bottom: 10px;
  text-align: center;
}

.task {
  background: white;
  padding: 12px;
  border-radius: 8px;
  margin-bottom: 10px;
  box-shadow: 0 1px 4px rgba(0,0,0,0.1);
}

.task button {
  margin-top: 10px;
  padding: 5px 10px;
  border: none;
  border-radius: 6px;
  background: #1976d2;
  color: white;
  cursor: pointer;
}

.chatgpt {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.chatgpt textarea {
  min-height: 100px;
}

```

```

.chatgpt button {
  width: 200px;
  padding: 10px;
  border: none;
  border-radius: 8px;
  background: #0d47a1;
  color: white;
  cursor: pointer;
  font-weight: bold;
}

.response {
  background: #f1f8e9;
  padding: 15px;
  border-radius: 8px;
  min-height: 80px;
  border: 1px solid #c5e1a5;
}

@media(max-width: 768px) {
  .swot-grid {
    grid-template-columns: 1fr;
  }

  .kanban {
    flex-direction: column;
  }
}
</style>
</head>
<body>

<h1>Painel Integrado - MPS.BR, SWOT e Kanban</h1>

<div class="container">

  <!-- SWOT -->
  <div class="card">
    <h2>Análise SWOT</h2>

```

```

<div class="swot-grid">
  <div class="swot-item forcas">
    <h3>Forças</h3>
    <textarea placeholder="Digite as forças..."></textarea>
  </div>

  <div class="swot-item fraquezas">
    <h3>Fraquezas</h3>
    <textarea placeholder="Digite as fraquezas..."></textarea>
  </div>

  <div class="swot-item oportunidades">
    <h3>Oportunidades</h3>
    <textarea placeholder="Digite as oportunidades..."></textarea>
  </div>

  <div class="swot-item ameacas">
    <h3>Ameaças</h3>
    <textarea placeholder="Digite as ameaças..."></textarea>
  </div>
</div>

<!-- MPS.BR -->
<div class="card">
  <h2>Controle de Processos - MPS.BR</h2>

  <table>
    <thead>
      <tr>
        <th>Processo</th>
        <th>Status</th>
        <th>Responsável</th>
      </tr>
    </thead>

    <tbody>
      <tr>
        <td>Gerência de Projetos</td>
        <td>
          <select>

```

```

        <option>Concluído</option>
        <option>Em andamento</option>
        <option>Pendente</option>
    </select>
</td>
<td>Equipe A</td>
</tr>

<tr>
<td>Garantia da Qualidade</td>
<td>
    <select>
        <option>Concluído</option>
        <option>Em andamento</option>
        <option>Pendente</option>
    </select>
</td>
<td>Equipe B</td>
</tr>

<tr>
<td>Gerência de Requisitos</td>
<td>
    <select>
        <option>Concluído</option>
        <option selected>Em andamento</option>
        <option>Pendente</option>
    </select>
</td>
<td>Equipe C</td>
</tr>
</tbody>
</table>
</div>

<!-- Kanban -->
<div class="card">
    <h2>Quadro Kanban</h2>

    <div class="kanban">

```

```

<div class="column" id="todo">
  <h3>A Fazer</h3>

  <div class="task">
    Desenvolver API
    <br>
    <button onclick="moveTask(this, 'doing') ">Mover</button>
  </div>

  <div class="task">
    Criar documentação
    <br>
    <button onclick="moveTask(this, 'doing') ">Mover</button>
  </div>
</div>

<div class="column" id="doing">
  <h3>Fazendo</h3>
</div>

<div class="column" id="done">
  <h3>Concluído</h3>
</div>

</div>
</div>

<!-- ChatGPT -->
<div class="card">
  <h2>Assistente ChatGPT</h2>

  <div class="chatgpt">
    <textarea id="prompt" placeholder="Digite uma pergunta para o
ChatGPT..."></textarea>

    <button onclick="consultarChatGPT()" ">
      Consultar ChatGPT
    </button>

    <div class="response" id="response">
      A resposta aparecerá aqui...

```

```

        </div>
    </div>
</div>

</div>

<script>
    function moveTask(button, targetColumnId) {
        const task = button.parentElement;
        const targetColumn = document.getElementById(targetColumnId);

        if (targetColumnId === 'doing') {
            button.setAttribute('onclick', "moveTask(this, 'done')");
        } else {
            button.innerText = 'Finalizado';
            button.disabled = true;
        }

        targetColumn.appendChild(task);
    }

    async function consultarChatGPT() {
        const prompt = document.getElementById('prompt').value;
        const responseDiv = document.getElementById('response');

        if (!prompt.trim()) {
            responseDiv.innerHTML = 'Digite uma pergunta.';
            return;
        }

        responseDiv.innerHTML = 'Consultando ChatGPT...';

        try {

            // EXEMPLO DE INTEGRAÇÃO
            // Substitua SUA_CHAVE_API pela chave da OpenAI

            const resposta = await
fetch('https://api.openai.com/v1/chat/completions', {
    method: 'POST',
    headers: {

```



```

        'Content-Type': 'application/json',
        'Authorization': 'Bearer SUA_CHAVE_API'
    },
    body: JSON.stringify({
        model: 'gpt-4.1-mini',
        messages: [
            {
                role: 'user',
                content: prompt
            }
        ]
    })
});

const dados = await resposta.json();

responseDiv.innerHTML = dados.choices[0].message.content;

} catch (erro) {
    responseDiv.innerHTML = 'Erro ao consultar a API.';
    console.error(erro);
}
}
</script>
</body>
</html>

```

Tela criada:

Painel Integrado - MPS.BR, SWOT e Kanban

Análise SWOT

Forças

Digite as forças...

Fraquezas

Digite as fraquezas...

Oportunidades

Digite as oportunidades...

Ameaças

Digite as ameaças...

Controle de Processos - MPS.BR

Processo	Status	Responsável
Gerência de Projetos	Em andamento	Equipe A
Garantia da Qualidade	Pendente	Equipe B
Gerência de Requisitos	Em andamento	Equipe C

Quadro Kanban

A Fazer

Fazendo

Concluído

Desenvolver API

EAD
EDUCAÇÃO A
DISTÂNCIA

UNIP
UNIVERSIDADE PAULISTA

Reflexões finais

Medir a maturidade do processo é essencial para uma organização compreender o seu estágio atual de eficiência para planejar um caminho de melhoria contínua.

Com a utilização da matriz SWOT como ferramenta de produtividade de software, a equipe pode visualizar quais são suas forças, fraquezas, oportunidades e ameaças, assim sabendo qual é o melhor caminho a se seguir e junto com o do Kanban no processo de desenvolvimento, podem limitar o tempo de cada tarefa, tendo a visão clara das que ainda precisam fazer e quais já foram feitas.

Com a utilização do ChatGPT como ferramenta inteligente, algumas tarefas podem ser feitas de maneira mais rápida e fácil, facilitando o desenvolvimento de software.

Referências

<https://promovesolucoes.com/quais-sao-os-niveis-de-maturidade-do-mps-br/>

<https://radar.ibegesp.org.br/como-a-analise-swot-pode-melhorar-a-estrategia-de-uma-organizacao/>

<https://www.atlassian.com/br/agile/kanban>

RELATÓRIO 5: Arquitetura de Sistemas em Camadas (Pacotes e Componentes) com Astah

Resumo teórico

Objetivo em se ter uma arquitetura em camadas do sistema de software.

Com a arquitetura em camadas o sistema tem uma estruturação em partes autônomas, sendo que cada uma detém responsabilidades e funcionalidades particulares, assim trazendo diversos benefícios, como reuso de código e simplificação da manutenção.

Arquitetura da informação no relacionamento entre pacotes.

A arquitetura da informação tem o objetivo de organizar e rotular websites, aplicativos digitais e conteúdo online, envolvendo a criação de uma estrutura lógica que facilite o encontro de informações pelos usuários para aumentar a usabilidade e a navegabilidade. Com os relacionamentos entre pacotes, podemos com a criação de módulos ou componentes planejar como o sistema irá se organizar e interagir com clareza, garantindo que a arquitetura da informação estará sendo aplicada de maneira eficiente.

Benefícios em se utilizar componentes como software de reúso.

Com o software de reúso reduzimos a duplicidade do código, assim facilitando a manutenção do sistema.

Arquitetura em camadas

O fundamento da arquitetura em camadas está na estruturação do sistema em camadas, cada uma delas possuindo responsabilidades e funcionalidades diferentes (camadas como: Apresentação, Lógica de Negócio e Acesso a Dados).

- **Camada de apresentação:** responsável pela interface do usuário.
- **Camada de lógica de negócios:** contém as regras de negócio da aplicação.
- **Camada de acesso a dados:** gerencia a comunicação com o banco de dados.

Essa arquitetura aumenta a segurança e escalabilidade, com ela mesmo que modifique uma camada, isso não impactaria todo o resto do sistema.

Figura 7 - Arquitetura em Camadas



Fonte:

<https://blog.grancursosonline.com.br/arquitetura-em-tres-camadas-para-aplicacoes-web/>

Diagrama de pacotes (DP)

Diagramas de pacotes são diagramas que vão mostrar a organização de vários elementos em formas de pacotes, esses pacotes são um agrupamento de elementos UML relacionados (como: diagramas, documentos e classes). Os elementos são colocados dentro dos pacotes, sendo representados como uma pasta de arquivo.

Diagramas das classes e do pacote:

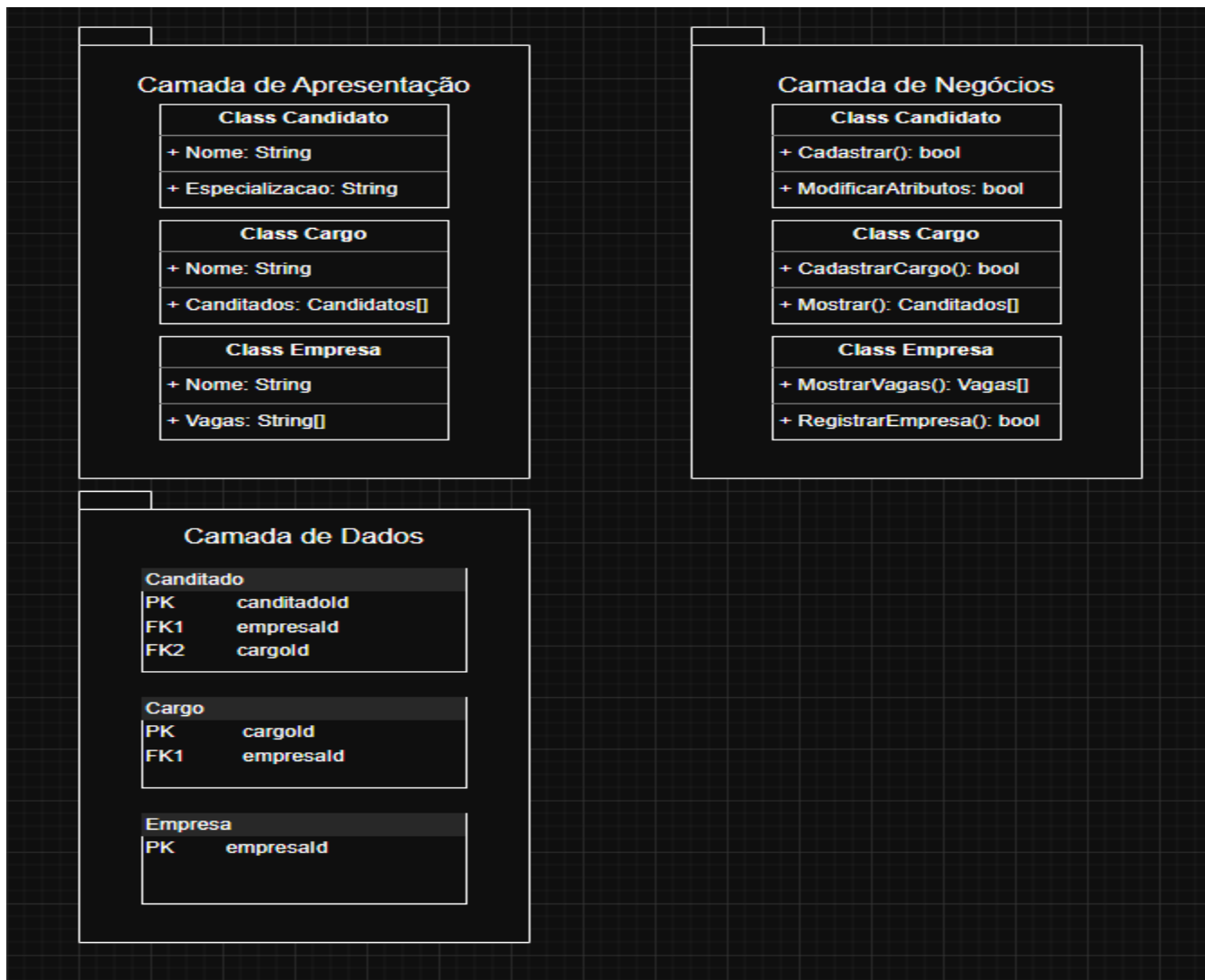
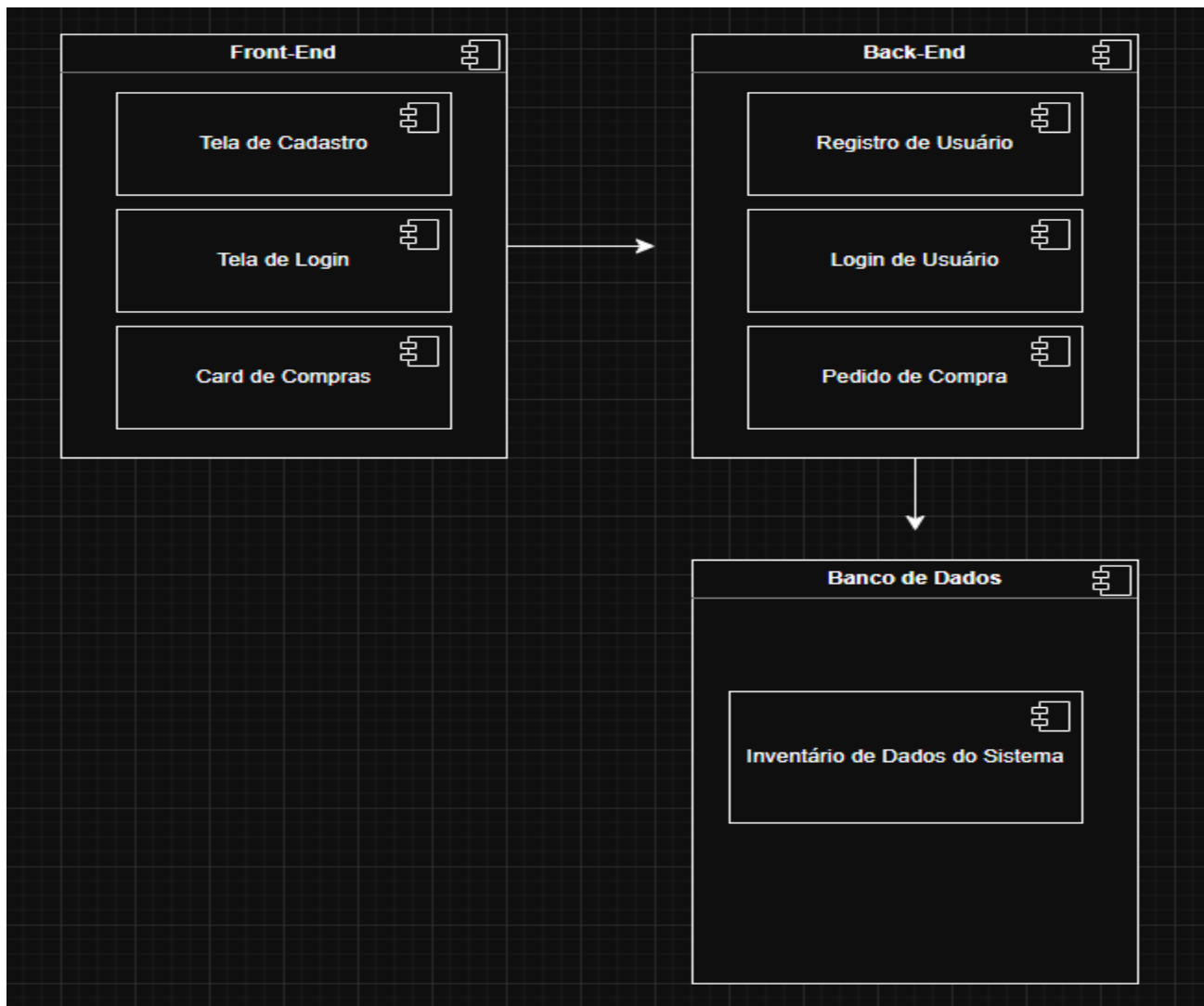


Diagrama de componentes (DCP)

O diagrama de componentes mostra o relacionamento entre diferentes componentes, um componente é um módulo de classe que irá representar sistemas independentes que possui a capacidade de interação com outros componentes do sistema.

Diagrama de componentes:



Reflexões finais

Entre os diversos benefícios em se trabalhar com arquitetura em camadas, podemos citar o aumento da segurança, escalabilidade e reuso de código que traz a simplificação da manutenção do sistema. E junto com o uso de diagramas de pacotes e de componentes, podemos construir o planejamento do sistema, assim se tornando mais fácil de se documentar e visualizar as suas funcionalidades, tendo a perspectiva de como ele será dividido entre as camadas.

Referências

https://dev.to/gabriel_ribeiro_a60b493db/arquitetura-em-camadas-um-padrao-essencial-para-sistemas-robustos-cjn

<https://www.conversion.com.br/blog/arquitetura-da-informacao/>

https://blog.infnet.com.br/arquitetura_software/o-que-e-e-como-fazer-arquitetura-de-software-e

m-camadas/

<https://www.lucidchart.com/pages/pt/diagrama-de-componentes-uml>

<https://www.lucidchart.com/pages/pt/diagrama-de-pacotes-uml>

RELATÓRIO 6: Modelagem de Arquitetura (Componentes) com Astah

Resumo teórico

Objetivo da estrutura em camadas e modelagem do processo de componentização.

A estrutura em camadas é um estilo arquitetural tem o objetivo de organizar as responsabilidades de partes de um software, cada camada tendo um isolamento e um propósito definido, sendo reutilizável ou substituível. Com a modelagem do processo de componentização fazemos a análise para a identificação das funcionalidades específicas de cada camada, assim organizando de uma melhor forma a estrutura da arquitetura do software.

Desenvolvimento estruturado de camadas pelo diagrama de componentes e suas ligações.

O diagrama de componentes facilita a visualização das conexões entre as camadas do software, fornecendo uma visão mais clara da arquitetura do sistema, auxiliando no desenvolvimento do software.

Os benefícios em se utilizar a componentização na arquitetura de sistemas.

Com a componentização na arquitetura de sistemas se tem uma menor duplicação de código, pois esses componentes são reutilizáveis, tornando o sistema mais fácil de se fazer manutenções, além de aumentar a escalabilidade e flexibilidade do sistema.

Aplicação da ferramenta Astah.

Com ferramentas iguais o Astah podemos criar diversos tipos de diagramas, agregando na criação de software de maneira intuitiva.

Estudo de caso

O sistema de estudo de caso escolhido foi um ERP para sistema de pedidos, com ela a empresa teria um sistema para organização, tendo acesso ao clientes registrados, pedidos no estoque e pedidos mais vendidos. Com a arquitetura em camadas esse sistema seria separado em 3 camadas, camada de aplicação, camada de lógica de negócio e camada de dados.

A camada de aplicação é o local em que a parte da interface do usuário fica, a camada

de lógica de negócio vai ter a parte do código fonte do back-end e na camada de dados vai possuir o banco de dados do sistema.

Requisitos do sistema de software

Os artefatos que serão construídos para o sistema vão ser diagramas de componentes para estruturar a arquitetura do software, trazendo uma representação visual do sistema.

Tabela de RF:

ID	Requisito Funcional (RF)	Componente da Aplicação (Camada de Apresentação/Interface)
RF01	O sistema deve permitir o cadastro, edição e exclusão de clientes.	Módulo de Gestão de Clientes
RF02	O sistema deve exibir uma lista de clientes registrados com opções de busca e filtro.	Tela de Consulta de Clientes
RF03	O sistema deve permitir o registro de pedidos associados a um cliente.	Formulário de Pedidos
RF04	O sistema deve exibir o estoque atualizado de produtos disponíveis.	Painel de Controle de Estoque
RF06	O sistema deve gerar relatório dos produtos mais vendidos em determinado período.	Relatório de Produtos Mais Vendidos
RF07	O sistema deve notificar o usuário sobre estoque baixo	Alertas da Interface

	de algum produto.	
RF08	O sistema deve permitir a visualização de detalhes de cada pedido (itens, quantidades, valor).	Detalhamento de Pedido na interface

Tabela de RS:

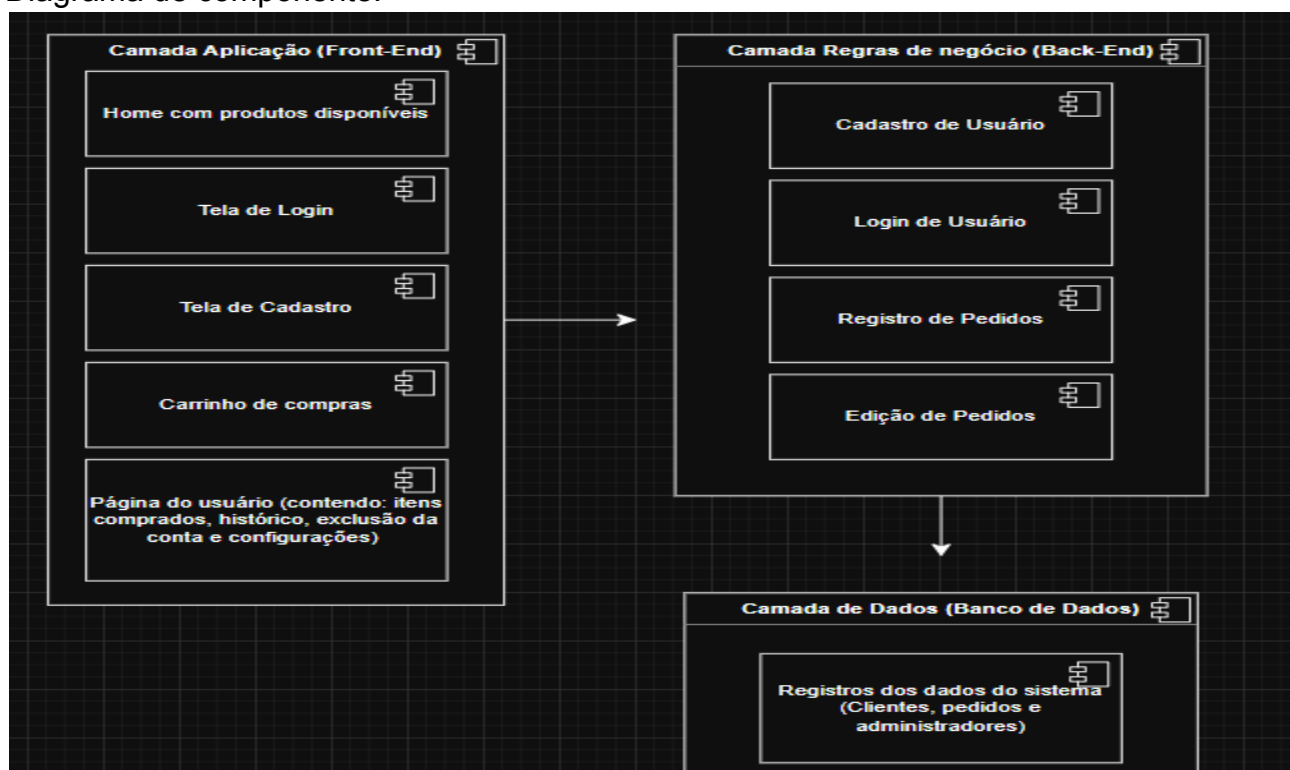
ID	Requisito Do Sistema	Componente do Sistema (Camadas Lógica e Dados)
RS01	O sistema deve garantir que o acesso a clientes e pedidos seja feito apenas por usuários autenticados.	Módulo de Autenticação e Controle de Acesso
RS02	A camada de dados deve assegurar a integridade referencial entre clientes, pedidos e produtos.	Banco de Dados Relacional (Constraints/FK)
RS03	A lógica de negócio deve calcular automaticamente o total do pedido com base nos itens e quantidades.	Regra de Negócio (Cálculo de Pedido)
RS04	O sistema deve armazenar logs de todas as alterações em pedidos e estoque.	Módulo de Auditoria
RS05	A recuperação de dados deve demorar no máximo 2 segundos para relatórios de até 10 mil registros.	Índices e Otimização de Consultas (Camada de Dados)

RS06	O sistema deve ser capaz de suportar até 100 usuários simultâneos sem perda de desempenho.	Gerenciamento de Sessões e Conexões com BD
RS07	A comunicação entre camada de aplicação e lógica de negócio deve ser criptografada.	API de Serviços com TLS/HTTPS
RS08	O sistema deve realizar backup automático diário da camada de dados.	Serviço de Backup do Banco de Dados

Artefatos desenvolvidos

No diagrama de componentes um componente é um módulo de classe que irá representar sistemas independentes, os artefatos que criamos foram diagramas que facilitam a representação visual da arquitetura de software escolhida, que foi a arquitetura em camadas, separando as camadas em componentes diferentes.

Diagrama de componente:



Reflexões finais

Entre os diversos benefícios em se trabalhar com arquitetura em camadas, podemos citar o aumento da escalabilidade, segurança e aumento na facilidade de manutenção dos componentes do sistemas. Com a visão do sistema do sistema em camadas, temos uma divisão por funcionalidade, construindo componentes independentes que são conectados uns com os outros. E sendo feito uma análise e aplicação da arquitetura de infraestrutura de TI, construímos sistemas complexos e robustos com uma maior eficiência.

Referências

<https://guia.dev/pt/pillars/software-architecture/layers.html#>

<https://edraw.wondershare.com.br/diagram-tips/component-diagram.html>

RELATÓRIO 7: Ecossistemas Digitais (SOA, Módulos, Implantação) com Astah

Resumo teórico

O que é um ecossistema digital.

Um ecossistema digital é uma rede complexa de pessoas, empresas e sistemas que utilizam a tecnologia para interagir entre si, aproveitando das camadas físicas, as camadas de informação e as camadas de aplicação para se comunicarem.

Arquitetura pela representação de seus nós

A visão arquitetural do ecossistema digital é fundamentada por uma rede de nós independentes que são conectados através de uma rede. Cada nó é uma parte do sistema, como banco de dados, sites, aplicativos ou módulos que executam uma funcionalidade específica.

Microserviços de TI

A arquitetura em microserviços é constituída por uma arquitetura nativa da nuvem que se baseia em uma aplicação composta por muitos componentes implementados de forma independente. Geralmente, possuem uma pilha de tecnologia, incluindo o banco de dados e um modelo para o gerenciamento de dados, se comunicam através de APIs REST e são organizados por capacidade empresarial.

Escalabilidade de um ecossistema digital.

A escalabilidade de um ecossistema digital está na capacidade do ecossistema suportar o aumento nas demandas, como aumento de usuários ou no volume de dados, sem perder desempenho.

Características das ferramentas de diagramação padrão UML.

As ferramentas de diagramação padrão UML tem a finalidade na criação de diagramas UML, sendo essenciais na estruturação da arquitetura de um software, facilitando o entendimento com uma visualização da estrutura criada para o sistema.

Estudo de caso

O ecossistema criado foi um microserviço em nuvem (DevOps). Os nós do sistema são:

- Usuários (Web/Mobile).
- API Gateway.
- Clusters (Kubernetes).
- Banco de Dados Distribuído.

Os artefatos criados foram scripts de build e deploy, microserviços de catálogo, de pagamentos e de usuários.

A arquitetura em sua totalidade foi baseada na arquitetura orientada a serviços (SOA) e a arquitetura para a aplicações específicas foi a de microserviços.

Infraestrutura do ecossistema de microserviços em nuvem

Os artefatos a serem produzidos para a infraestrutura do ecossistema de microserviços em nuvem são: Scripts, diagramas de componentes, diagramas de implantação e documentação dos requisitos funcionais e não funcionais do sistema.

Item	Nó do Sistema	Nó do Sistema	Nó do Sistema
1	Usuários (Web/Mobile)	Microserviço de catálogo	Consumo de serviço via interface padronizada (SOA expõe serviços a múltiplos canais)
2	Usuários (Web/Mobile)	Microserviço de usuários	Interação com serviço de autenticação/perfil, seguindo o acoplamento flexível da SOA
3	API Gateway	Scripts de build e deploy (CI/CD)	Ponto único de entrada orquestrando chamadas a serviços, típico do padrão SOA com

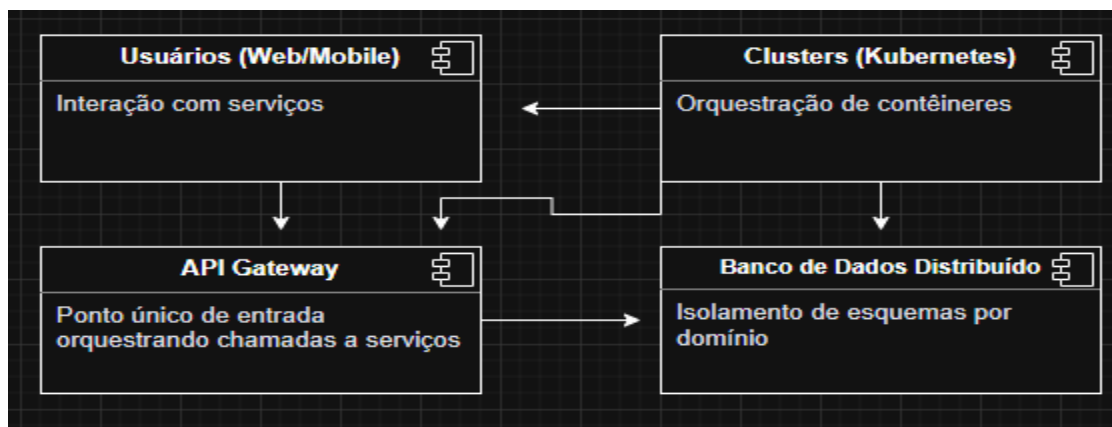
			gateway
4	Clusters (Kubernetes)	Microserviço de pagamentos	Escalabilidade e descoberta de serviço (SOA na prática com orquestração de contêineres)
5	Banco de Dados Distribuído	Microserviço de catálogo	Persistência independente por serviço, alinhado ao princípio SOA de autonomia de dados
6	Banco de Dados Distribuído	Microserviço de usuários	Isolamento de esquemas por domínio, promovendo reusabilidade e baixo acoplamento
7	API Gateway	Microserviço de pagamentos	Roteamento e mediação de chamadas síncronas/assíncronas, característica forte da SOA
8	Clusters (Kubernetes)	Scripts de build e deploy	Automação do ciclo de vida dos serviços, suportando a governança e evolução contínua na

			SOA
--	--	--	-----

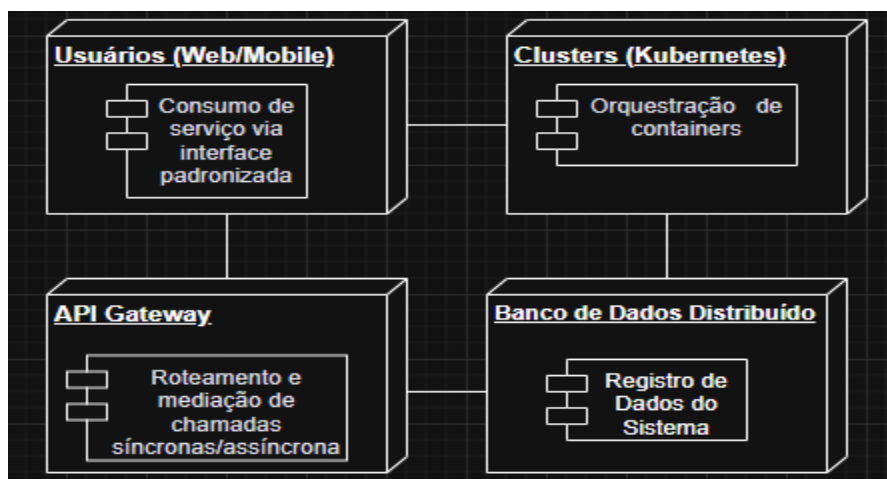
Artefatos desenvolvidos

O mapeamento dos módulos de infraestrutura com os artefatos desenvolvidos segue a lógica de que cada nó de infraestrutura é responsável por uma funcionalidade específica, como hospedar, orquestrar ou dar acesso a um ou mais microsserviços. O API Gateway atua como ponto de entrada único, roteando requisições dos usuários para os microsserviços de catálogo, pagamentos e usuários. Os clusters Kubernetes gerenciam a implantação, utilizando os scripts de build e deploy para automatização do ciclo de vida do sistema. Já o banco de dados distribuído é mapeado de forma isolada por domínio, garantindo que cada microsserviço (catálogo, usuários, pagamentos) possua sua própria camada de persistência.

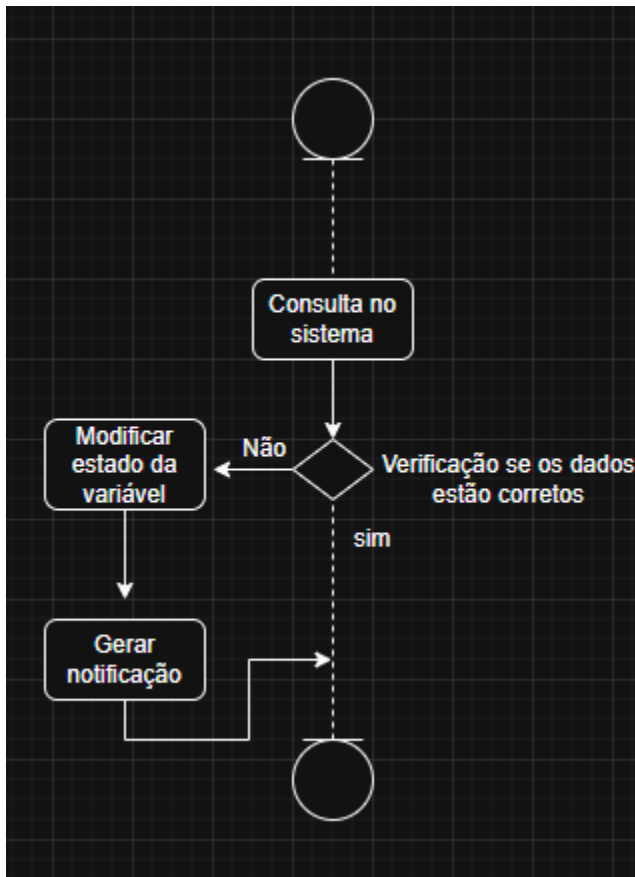
- Diagrama componentes:



- Diagrama implantação:



- Diagrama notificações:



Reflexões finais

Os benefícios da SOA e microsserviços em ecossistemas digitais trazem são: menor duplicação de código, maior escalabilidade, segurança e maior facilidade para fazer manutenções no sistema.

A análise e aplicação da arquitetura de infraestrutura de TI são elementos essenciais para a construção de sistemas robustos e confiáveis, sem isso essa tarefa seria ainda mais complicada e difícil de ser aplicada.

Referências

https://www-imd-org.translate.goog/blog/digital-transformation/digital-ecosystems/?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

<https://www.ibm.com/br-pt/think/topics/microservices>

<https://www.redhat.com/pt-br/topics/cloud-native-apps/what-is-service-oriented-architecture>

RELATÓRIO 8: Caso de Testes de V&V com GanttProject

Aluno: _____

Turma: _____ Data: ____/____/____

1. Resumo teórico

Escreva aqui um texto de 8 a 10 linhas abordando:

- Compreensão sobre o processo de verificação e validação (V&V).
- Utilização do cronograma e rede de cartões (kanban) no controle da V&V.
- Entendimento sobre o método PERT para a construção do cronograma.
- Características das ferramentas de gestão de projetos como o GanttProject (ou Microsoft Project).

2. Estudo de caso

Escreva aqui um texto de 8 a 10 linhas sobre a técnica de avaliação e revisão de programas (PERT) para atividades de V&V.

- Atividades do V&V e respectivas tarefas – as principais atividades do V&V são: planejamento da V&V; verificação; caso de testes (unitário, integração, aceitação e regressão); validação com stakeholder e retrospectiva (qualidade e encerramento).

3. Tabela PERT

- De 3 a 5 linhas, descreva as principais características da tabela PERT.
- Construa uma tabela com as colunas ID, Tarefa, Dependências e Período com pelo menos 12 itens.
(Inserir tabela PERT).

4. Cronograma e kanban

- De 3 a 5 linhas, explique a implementação da tabela PERT, bem como o cronograma e rede de cartões a ser gerado.
- Construa o cronograma e rede de cartões com uma ferramenta de gestão de projetos (p. ex.: GanttProject).

(1. Inserir imagem de tela (com legenda) das tarefas e cronograma).

(2. Inserir imagem de tela (com legenda) da rede de cartões (Gráfico PERT)).

5. Reflexões finais

- Benefícios da gestão de projetos e quadro kanban no processo V&V.
- Análise dos artefatos produzidos: tabela PERT implementada, cronograma e kanban.

6. Referências

Inclua pelo menos uma referência do livro-texto e outras fontes utilizadas para a elaboração deste relatório.

RELATÓRIO 9: Gestão de Tarefas FDD em Kanban com ChatGPT

Aluno: _____

Turma: _____ Data: ____/____/____

1. Resumo teórico

Escreva aqui um texto de 8 a 10 linhas abordando:

- Compreensão sobre as operações de design e comportamento (FDD).
- Utilização da rede de cartões (kanban) no fluxo contínuo e controle do progresso das tarefas.
- Entendimento do ChatGPT como ferramenta inteligente para geração de código para a aplicação.

2. Estudo de caso

Escreva aqui um texto de 8 a 10 linhas sobre:

- FDD e a organização de trabalhos entregáveis de pequenas funcionalidades.
- Uso do ChatGPT como um facilitador inteligente.

3. Etapas do processos do FDD

- De 8 a 10 linhas, descreva as cinco principais etapas do FDD: 1. Modelagem do domínio de requisitos (FDD); 2. Design orientado a domínio DDD (diagrama da UML a ser usado); 3. Automação de testes por funcionalidade; 4. Abordagem orientada a testes (aceite, comportamento e refatoração); 5. Gestão do processo de negócio e integração contínua.

4. Utilização do ChatGPT

- De 3 a 5 linhas, explique o ChatGPT no auxílio à criação de conteúdos, automação de tarefas e geração de aplicação.
- Elaborar “pergunta” ao ChatGPT para codificação, com a implementação de um quadro Kanban para o controle das tarefas FDD.
- Codificação em HTML.
- Codificação em JS.

(1. Inserir código gerado em HTML para bloco de notas e imagem de tela da execução da aplicação).

(2. Inserir código gerado em JS para bloco de notas e imagem de tela da execução da aplicação).

5. Reflexões finais

- Benefícios do uso de um modelo de linguagem baseado em inteligência artificial (IA) no desenvolvimento de sistemas de software.
- Análise dos artefatos produzidos.

6. Referências

Inclua pelo menos uma referência do livro-texto e outras fontes utilizadas para a elaboração deste relatório.

RELATÓRIO 10: Versionamento com Draw.io

Aluno: _____

Turma: _____ Data: ____/____/____

1. Resumo teórico

Escreva aqui um texto de 8 a 10 linhas abordando:

- Explique o que é versionamento de software.
- Utilização da rede de cartões (kanban) no fluxo contínuo e controle de versões e releases.
- Utilização de ferramenta para diagramação, estruturas e modelos em geral.

2. Estudo de caso

Escreva aqui um texto de 8 a 10 linhas sobre:

- Versionamento no controle de mudanças (p. ex.: em de sites de fornecedores de aplicações como: sistemas de software ERP da IBM, TOTVS, SAP e outros).
- O uso do modelo padrão (X.YZzzzz) que determina as mudanças de: estrutura, função e patch).
- Necessidade da diagramação e de um grafo de controle funcional.

3. Tabela de mudanças

- De 3 a 5 linhas, descreva o controle de mudanças que atenda ao padrão (X.YZzzzz), situando o que são os tipos de estrutura, função e patch.
(Inserir tabela (com legenda) de versionamento com as colunas: Item, Versão, Tipo e Descrição da Mudança).

4. Diagrama funcional com Draw.io

- De 3 a 5 linhas, explique a ferramenta Draw.io na criação de quadros e modelos utilizáveis. (No Draw.io em “Geral”, utilize os blocos “Container” e “List” para o mapeamento das mudanças).

(Inserir aqui a imagem (com legenda) do quadro funcional gerado)

5. Reflexões finais

- Benefícios do uso do versionamento do software.
- Análise dos artefatos produzidos.

6. Referências

Inclua pelo menos uma referência do livro-texto e outras fontes utilizadas para a elaboração deste relatório.